

本科毕业设计（论文）

论文题目 一种即插即用的联邦学习特征蒸馏模块
设计与实现

作者姓名 朱炳硕

专 业 软件工程

指导教师 郭丁丁教授

2026 年 6 月

燕山大学本科毕业设计（论文）

一种即插即用的联邦学习特征蒸馏模块设计与实现

学 院： 人工智能学院
专 业： 软件工程
姓 名： 朱炳硕
学 号： 202211130341
指 导 教 师： 郭丁丁
答 辩 日 期： □□□□年□月

学位论文原创性声明

郑重声明：所呈交的学位论文《一种即插即用的联邦学习特征蒸馏模块设计与实现》，是本人在导师的指导下，独立进行研究取得的成果。除文中已经注明引用的内容外，本论文不包括他人或集体已经发表或撰写过的作品成果。对本文的研究

做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果，并承诺因本声明而产生的法律结果由本人承担。

学位论文作者签名：

日期： 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权燕山大学将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

保 密，在__年解密后适用本授权书。

本学位论文属于

不保密。

(请在以上相应方框内打“√”)

学位论文作者签名：

日期： 年 月 日

指导教师签名：

日期： 年 月 日

摘 要

摘要是论文内容的高度概括，应具有独立性和自含性，即不阅读论文的全文，就能获得必要的信息。摘要应包括本论文的目的、主要研究内容、研究方法、创造性成果及其理论与实际意义。摘要中不宜使用公式、化学结构式、图表和非公知公用的符号与术语，不标注引用文献编号，同时避免将摘要写成目录式的内容介绍。

关键词：关键词 1；关键词 2；.....；.....；关键词 5；
关键词 6

Abstract

The rehabilitation mechanism is the foundation for a rehabilitation robot to realize its motion, and the quality of the rehabilitation mechanism decides the rehabilitation effect of patients with the rehabilitation robot. The research of rehabilitation mechanism is.....

Keywords: keyword 1; keyword 2; keyword 3;;;
keyword 6

目 录

摘 要	I
Abstract	II
第1章 绪 论	1
1.1 课题背景概述.....	1
1.2 现有解决方法.....	1
1.3 局限性.....	2
第2章 相关理论与方法基础	3
2.1 联邦学习.....	3
2.2 FedAvg.....	3
2.3 FedProx.....	3
2.4 FedNova.....	4
2.4 SCAFFOLD.....	4
2.5 数据蒸馏与特征分离思想.....	4
2.7 FedFed.....	5
第3章 系统总体设计与数据异构的实现	6
3.1 系统设计目标.....	6
3.2 系统总体架构与插件化设计.....	6
3.3 本章小结.....	6
第4章 FedFed 特征蒸馏插件设计与实现.....	7
4.1 FedFed 插件模块组成.....	7
4.2 图像空间特征蒸馏方法.....	7
4.3 两阶段插件训练机制.....	11
4.4 共享样本池设计与辅助信息交互.....	11
4.5 本章小结.....	13
第5章 实验设计与性能评估	14
5.1 实验目的.....	14
5.2 实验环境与基础设置.....	14
5.3 评价指标与公平性控制.....	14
5.4 FedFed 插件主性能验证.....	15
5.5 主性能改进历程分析.....	18
5.6 插件关键模块消融实验.....	18
5.7 本章小结.....	20
第6章 机制验证与工程分析	22
6.1 特征蒸馏机制诊断.....	22
6.2 隐私验证.....	23
6.3 跨联邦优化器可插拔验证.....	28
6.4 训练开销分析.....	30

6.5 本章小结.....	31
结 论	32
参考文献	33
致 谢	35

第1章 绪 论

1.1 课题背景概述

随着移动互联网和物联网设备的快速发展，大量数据分散存储在用户终端设备中。传统的集中式机器学习需要将数据集中上传到服务器进行统一训练，但这种方式在现实应用中面临严重的隐私泄露风险，同时也可能违反数据保护相关法规。为解决这一问题，Google 于 2017 年提出了联邦学习（Federated Learning）框架[1]，该框架允许各客户端在本地利用自身数据训练模型，仅将模型参数上传至服务器进行聚合，从而在不直接共享原始数据的情况下实现协同学习。

然而在实际应用场景中，不同客户端的数据往往具有显著差异，这种现象被称为数据异构或 Non-IID（Non-Independent and Identically Distributed）问题[2]。例如，不同用户设备产生的数据类别比例不同、采集环境不同或存在明显的分布偏移，这会导致客户端模型在本地训练时优化方向不一致，从而产生所谓的“客户端漂移（Client Drift）”问题，使得全局模型的收敛速度和泛化能力明显下降。针对这一问题，国内外学者提出了多种改进方法。

1.2 现有解决方法

目前已有研究主要集中在以下几个方向：第一类是优化算法改进方法，例如 FedProx[3]、SCAFFOLD[4]、FedNova[5] 等算法通过引入额外的正则化项或梯度校正机制来减少局部模型更新偏差，从而缓解数据异构带来的影响。第二类是个性化联邦学习方法，例如 FedSPU[6]、Look Back for More[7] 等方法通过在模型中引入个性化参数或利用历史更新信息，使客户端模型能够更好地适应本地数据分布。第三类方法是结构或表示共享方法，例如在联邦图学习领域中提出的虚拟节点（Virtual Nodes）[8]或原型共享（Prototype Sharing）[9]机制，通过共享部分结构化信息实现跨客户端表示对齐。第四类是近年来提出的特征蒸馏与信息分解方法，通过共享不

具备隐私风险的敏感特征来缓解数据异构问题，例如，NeurIPS 2023 提出的 FedFed 方法通过数据蒸馏机制将原始数据划分为性能敏感特征和性能鲁棒特征，其中敏感特征包含与标签预测密切相关的信息，鲁棒特征包含剩余的自身隐私信息。各个客户端仅共享不含隐私信息的敏感特征，提升联邦学习性能与稳定性[10]。

1.3 局限性

目前解决方法众多，联邦学习算法更新很快，但大多都是耦合进联邦学习基线中，非常不利于及时迁移，也不利用实际应用部署。

FedAvg、FedProx、FedNova、SCAFFOLD 等方法的主流程不同，但都存在客户端本地训练、服务器聚合、下一轮广播这些共同环节。

第2章 相关理论与方法基础

2.1 联邦学习

联邦学习是一种面向多客户端数据协同建模的分布式机器学习范式，其核心思想是在不直接汇聚各客户端原始数据的前提下，由服务器协调多个客户端共同训练全局模型。典型流程包括：服务器初始化全局模型并下发给被选中的客户端；客户端基于本地私有数据进行若干轮本地训练；客户端将模型参数或模型更新上传至服务器；服务器根据各客户端样本数量或更新权重进行聚合，得到新的全局模型；上述过程循环执行，直至模型收敛或达到预设通信轮数。该机制能够降低原始数据集中存储带来的隐私风险和通信成本，但也面临数据异构、设备异构、通信不稳定等问题，其中 Non-IID 数据分布是影响联邦模型性能的重要因素之一[11]。

2.2 FedAvg

FedAvg 是联邦学习中最基础、应用最广泛的优化算法之一。该算法在每一轮通信中随机选取部分客户端参与训练，各客户端从服务器接收当前全局模型后，在本地数据上执行多步随机梯度下降，然后将训练后的模型参数上传至服务器。服务器按照客户端本地样本数量对模型参数进行加权平均，形成下一轮全局模型。与每一步梯度都进行通信的 FedSGD 相比，FedAvg 通过增加客户端本地训练步数显著减少通信频率，因此具有较高的通信效率[12]。但是，在客户端数据分布高度异构时，本地模型更新方向可能存在明显偏移，简单加权平均容易导致全局模型收敛变慢或精度下降。

2.3 FedProx

FedProx 可以看作是对 FedAvg 的一种稳健化改进，主要用于缓解联邦学习中的系统异构和统计异构问题。FedProx 在客户端本地目标函数中加入近端正则项，使本地模型在训练过程中不要过度偏离当前全局模型。其本地优化目标通常可表示为原始经验风险项加上 $\mu/2 ||w - w_t||^2$ ，其中 w_t 为当前轮次下发的全局模

型参数， μ 控制近端约束强度。该设计使客户端即使在数据分布差异较大或本地训练步数不一致的情况下，也能保持相对稳定的更新方向，从而提高异构场景下的训练稳定性[13]。因此，FedProx 很适合作为本文插件化适配实验中的第二个基线算法。

2.4 FedNova

FedNova 主要针对异构联邦优化中的“目标不一致”问题。FedAvg 在不同客户端本地训练步数不一致时，直接对本地模型结果进行加权平均，可能导致全局模型实际优化的目标函数与原始联邦优化目标不一致。FedNova 通过对客户端本地更新进行归一化处理，使服务器聚合时考虑不同客户端实际执行的本地更新量，从而减轻由本地步数差异带来的聚合偏差[14]。与 FedProx 通过修改本地目标函数增强稳定性不同，FedNova 更关注服务器端聚合更新的规范化问题，体现了从“聚合规则”角度处理异构性的思路。

2.4 SCAFFOLD

SCAFFOLD 针对 Non-IID 数据下客户端更新方向偏移的问题，引入控制变量对本地更新进行校正。其基本思想是分别维护服务器端和客户端的控制变量，用于估计并修正本地梯度与全局梯度之间的偏差，从而降低 client drift 对全局收敛的影响。与 FedAvg 直接聚合本地模型不同，SCAFFOLD 通过控制变量显式约束客户端更新方向，使其在理论上能够更好地应对数据异构和客户端采样带来的不稳定性[15]。该类方法说明，Non-IID 问题不仅可以从数据共享或特征共享角度解决，也可以从优化校正角度进行处理。

2.5 数据蒸馏与特征分离思想

传统意义上的数据蒸馏通常指将大规模训练数据压缩为少量具有代表性的合成样本，使模型在这些小规模蒸馏数据上训练后仍能接近原始数据集训练效果[16]。但本文所讨论的 FedFed 图像插件中的“蒸馏”更接近 FedFed 论文中的特征蒸馏与数据特征分离思想，并不是常规的教师-学生知识蒸馏。FedFed 借助生成器 $q(x)$ 将原始输入样本 x 分解为性能鲁棒特征 $x_r = q(x)$ 和性能敏感特征 $x_s = x - q(x)$ ，其中性能敏感特征包含对分类性能更关键的信息，可在服务器端共享以缓解客户端间的

数据异构，而性能鲁棒特征则保留在本地，以降低直接共享原始数据带来的隐私风险[18]。在实现上，本文主线采用 Beta-VAE 类生成器结构，其理论基础与变分自编码器中的潜变量建模和重参数化训练方法有关[17]。

2.7 FedFed

FedFed 是本插件项目所参考的方法，它的核心目标是在隐私保护和模型性能提升之间取得折中：不直接共享客户端原始图像，而是共享从原始图像中提取出的性能敏感特征。其整体流程可以分为两个阶段。第一阶段为特征蒸馏阶段，客户端训练生成器 $q(x)$ 和蒸馏分类器，使 $x_s = x - q(x)$ 具有较强的类别判别能力，同时通过重构损失、KL 损失和范数约束限制特征分离过程，避免敏感特征退化为原始图像复制。客户端按类别筛选并上传部分性能敏感特征，服务器将其维护为共享特征池。第二阶段为正式联邦训练阶段，服务器向客户端下发全局模型和共享敏感特征，客户端同时利用本地数据和共享特征参与训练，再将模型更新上传给服务器聚合。FedFed 论文证明了共享性能敏感特征能够缓解数据异构对联邦训练的影响[18]。本文基于其官方代码结构和算法流程进行工程化重构，将生成器训练、敏感特征上传、共享特征池维护和共享特征辅助训练封装为 `fedfed_image_plugin`，从而实现 FedFed 机制在联邦学习框架中的插件化接入[19]。

图 1 FedFed 方法特征分离机制示意图

图 2 FedFed 工作流程概述示意图

第3章 系统总体设计与数据异构的实现

3.1 系统设计目标

本文系统面向异构联邦图像分类任务，目标是在不共享客户端原始数据的前提下，将 FedFed 的特征蒸馏思想封装为一个即插即用模块，并接入 FedAvg 训练框架。

系统需要同时满足两个要求。第一，保留 FedAvg 的标准训练流程，包括客户端选择、模型下发、本地训练、参数上传和服务器聚合。第二，将 FedFed 算法中的：特征蒸馏、性能敏感特征提取、共享样本池构建和共享样本辅助训练从主流程中解耦，使其作为独立插件运行。

因此，本文系统不是重新设计一个完整联邦学习框架，而是在已有 FedAvg 骨架上增加插件接口，使 FedFed 模块能够以较低侵入方式接入和关闭。

3.2 系统总体架构与插件化设计

系统总体架构如图所示，整体由三部分组成：**FedAvg** 基线骨架层、插件协议与挂接层、FedFed 插件实现层。系统以 FedAvg 作为主训练通道，插件模块只通过额外辅助信息参与训练过程，通过插件协议和接口接入 FedAvg 基线，不改变 FedAvg 的模型广播、客户端本地训练和服务器参数聚合方式。

FedAvg 基线骨架层负责完成联邦学习的基本训练流程。服务器端训练模块负责选择参与客户端、下发全局模型、接收客户端模型参数并完成加权聚合；客户端训练模块负责接收全局模型，在本地数据上训练，并向服务器上传模型参数和本地样本数。当插件关闭时，系统退化为标准 FedAvg 训练过程。

插件协议与挂接层位于 FedAvg 主流程和具体插件实现之间。该层首先根据实验配置判断是否启用插件，并选择需要加载的插件类型以实现插件的灵活启停。随后，系统分别在客户端侧和服务器侧建立插件挂接点来实现目标基线文件对插件的接入。

FedFed 插件实现位于最外层，作为具体的特征蒸馏模块被插件协议层调用。客

户端 FedFed 模块负责从本地数据中提取可共享信息，并将其打包为辅助信息上传给服务器；服务器 FedFed 模块负责接收各客户端上传的辅助信息，整合形成全局共享内容，并在后续训练轮次中下发给客户端辅助本地训练。

从数据通道看，系统存在两类信息流。实线表示 FedAvg 主模型通道，包括服务器下发全局模型、客户端上传模型参数和样本数；虚线表示插件信息通道，包括客户端上传辅助信息、服务器下发全局辅助信息以及插件与主训练流程之间的挂接关系。二者相互配合，但职责分离：FedAvg 通道保证基础联邦训练过程，插件通道提供额外特征蒸馏能力。该设计使 FedFed 模块能够以低侵入方式接入 FedAvg，同时保留插件关闭后的基线可比性。

3.3 本章小结

再.

.....

第 4 章 FedFed 特征蒸馏插件设计与实现

4.1 FedFed 插件模块组成

FedFed 插件由客户端侧模块、服务端侧模块和生成器模块三部分组成。三者分别负责本地特征提取、全局状态维护和图像空间特征分离，共同完成 FedFed 特征

蒸馏机制。

客户端侧模块运行在每个客户端本地，主要包括主分类模型、生成器、分类器、敏感特征提取单元和辅助信息构造单元。主分类模型用于正式联邦训练，并在训练结束后向服务器上传本地模型参数。生成器用于对本地图像进行重构得到性能鲁棒特征，并与原始图像相减得到性能敏感特征。分类器复用主分类模型结构，用于约束敏感特征具有类别判别能力。敏感特征提取单元负责根据生成器输出计算可共享特征。辅助信息构造单元则将生成器状态、分类器状态、敏感特征及其标签打包为插件辅助信息，随客户端更新一起上传给服务器。

服务端侧模块负责维护插件的全局状态。其核心包括插件状态聚合、共享样本池维护和辅助信息广播构造。插件状态聚合单元接收客户端上传的生成器状态和分类器状态，并按照客户端样本数量进行加权聚合，得到全局生成器状态和全局分类器状态。共享样本池维护单元接收客户端上传的性能敏感特征，按类别保存为共享样本集合。辅助信息广播构造单元根据当前服务器插件状态，向客户端下发生成器状态和共享样本集，使客户端能够在后续训练中使用全局共享信息。

生成器模块是客户端侧特征分离的核心。本文采用 Beta-VAE 风格生成器，其结构包括编码器、潜变量空间和解码器。编码器通过卷积和残差块对输入图像进行压缩，潜变量空间输出均值和方差并进行重参数采样，解码器再将潜变量恢复到图像空间。生成器输出作为性能鲁棒特征，原始图像与生成器输出的鲁棒特征之间的差值作为性能敏感特征。

4.2 图像空间特征蒸馏方法

4.2.1 蒸馏方法概述

FedFed 插件在图像空间中进行特征分离。客户端首先利用生成器对本地图像进行重构，生成器输出被视为性能鲁棒部分，原始图像与重构结果之间的差值被视为性能敏感部分。性能鲁棒部分主要保留图像主体结构和冗余信息，性能敏感部分则保留与分类任务更相关的判别信息。

为实现该过程，插件采用 Beta-VAE 风格生成器。该生成器由编码器、潜变量空间和解码器组成。编码器压缩输入图像，潜变量空间对隐变量分布进行约束，解码器将潜变量恢复到图像空间。与普通自编码器相比，Beta-VAE 通过潜变量正则限制

表示容量，使生成器更倾向于保留图像主体信息，而将分类相关的差异信息留在敏感特征中。

插件还引入蒸馏分类器对性能敏感特征进行监督。蒸馏分类器接收敏感特征并预测其类别，用于约束敏感特征保留分类判别能力。由此，生成器和蒸馏分类器共同构成特征蒸馏网络：生成器负责分离图像信息，蒸馏分类器负责保证分离出的敏感特征对分类任务有效。

在训练阶段，系统加入随机裁剪、水平翻转、Mixup 和 Mosaic 等增强方式，以提高生成器和蒸馏分类器的鲁棒性。该增强仅作用于特征蒸馏阶段，不改变正式联邦训练中的模型聚合方式。

4.2.2 特征蒸馏网络结构设计

特征蒸馏网络由生成器和分类器组成。生成器负责从原始图像中生成性能鲁棒特征，蒸馏分类器负责约束性能敏感特征的类别判别能力。

本文采用 Beta-VAE 风格生成器。其结构包括编码器、潜变量层和解码器。编码器将输入图像压缩为潜在表示，潜变量层通过均值和方差建模隐空间分布，解码器将潜变量恢复到图像空间。生成器输出记为性能鲁棒特征，原始图像与生成器输出的差值记为性能敏感特征。

Beta-VAE 的作用不是单纯重构图像，而是在有限潜变量容量下保留图像主体信息。通过这种约束，生成器倾向于保留对分类贡献较弱的鲁棒部分，而差值部分保留更多与标签相关的判别信息。

分类器接收性能敏感特征并预测其标签。若敏感特征能够被正确分类，说明其中包含对模型性能有贡献的信息。因此，生成器和蒸馏分类器共同实现特征分离：重构约束保证鲁棒特征有效，分类约束保证敏感特征可用，潜变量约束保证生成过程稳定。

4.2.3 特征蒸馏目标设计

特征蒸馏阶段的目标是学习一个生成器，使其能够从原始图像中分离出性能敏感特征；同时训练蒸馏分类器，使该敏感特征保留类别判别信息。本文采用的蒸馏目标函数为：

$$L_{distill} = \lambda_{fd} CE(C(x_s), y) + \lambda_{re}(t) \|G(x) - x\|_2^2 + \lambda_x CE(C(x), y) + \beta L_{kl}$$

其中, x 表示客户端本地输入图像, y 表示对应类别标签, $G(\cdot)$ 表示生成器, $C(\cdot)$ 表示分类器。生成器输出 $G(x)$ 被视为性能鲁棒特征, 性能敏感特征定义为:

$$x_s = x - G(x)$$

第一项为性能敏感特征分类损失:

$$\mathcal{L}_{fd} = CE(C(x_s), y)$$

该项要求蒸馏分类器能够根据性能敏感特征预测真实标签。它直接约束 x_s 保留与分类任务相关的信息, 是特征蒸馏阶段的核心监督信号。

第二项为图像重构损失:

$$\mathcal{L}_{re} = \|G(x) - x\|_2^2$$

该项要求生成器输出尽量接近原始图像, 用于保证性能鲁棒特征保留图像主体结构。若缺少该约束, 生成器输出可能退化为无意义结果, 从而影响敏感特征的稳定分离。本文采用随训练阶段变化的重构权重 $\lambda_{re}(t)$, 在蒸馏初期加强重构约束, 使生成器先获得稳定的图像表达能力。

第三项为原图分类损失:

$$\mathcal{L}_x = CE(C(x), y)$$

该项用于增强蒸馏分类器对原始图像的基础判别能力。由于敏感特征由原图与生成器输出相减得到, 其分布在训练初期并不稳定, 因此引入原图分类监督可以提高分类器训练稳定性。

第四项为潜变量正则项:

$$\mathcal{L}_{kl} = -\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

其中, μ 和 σ^2 分别表示 Beta-VAE 潜变量分布的均值和方差。该项用于约束潜变量分布接近标准正态分布, 避免生成器隐空间过度自由, 从而提高特征分离过程的稳定性。

各权重参数用于控制不同目标之间的相对强度。其中, λ_{fd} 控制敏感特征分类约束, $\lambda_{re}(t)$ 控制图像重构约束, λ_x 控制原图分类约束, β 控制潜变量正则强度。

此外, 系统在特征蒸馏阶段引入随机裁剪、水平翻转、Mixup 和 Mosaic 等数据增强策略。随机裁剪和水平翻转增加输入变化; Mixup 用于增强蒸馏分类器的鲁棒性; Mosaic 用于提高生成器在复杂图像组合上的重构能力。这些增强仅作用于特征蒸馏阶段, 不改变正式 FedAvg 训练中的模型聚合方式。

4.3 两阶段插件训练机制

FedFed 插件采用两阶段训练方式。

第一阶段是特征蒸馏阶段。该阶段在正式 FedAvg 训练前执行。服务器选择部分客户端参与特征蒸馏，并向客户端下发当前生成器和分类器状态。客户端在本地数据上训练生成器和分类器，使生成器能够提取性能敏感特征，使分类器能够识别这些特征对应的类别。客户端训练结束后，将生成器和分类器状态上传给服务器。服务器按照客户端样本数量进行加权聚合，得到新的全局插件状态。

特征蒸馏阶段结束后，系统进入共享特征收集过程。服务器要求所有客户端使用训练后的生成器提取本地性能敏感特征。客户端将提取到的敏感特征及其标签上传给服务器。服务器按类别整理这些特征，形成全局共享样本池。

第二阶段是正式联邦训练阶段。该阶段恢复标准 FedAvg 训练流程。服务器每轮下发全局模型，同时将共享特征池中的特征下发给客户端。客户端训练本地模型时，将本地数据与共享性能敏感特征一起用于分类训练。训练结束后，客户端只需要上传本地模型参数和样本数量，服务器继续按照 FedAvg 方式进行聚合。

通过两阶段设计，系统将“特征蒸馏能力学习”和“共享特征辅助训练”分离开来。前者负责构建可共享信息，后者负责利用共享信息提升联邦训练效果。

4.4 共享样本池设计与辅助信息交互

FedFed 插件在服务器端维护共享样本池，用于保存各客户端上传的性能敏感特征。共享特征池不是原始数据集合，而是由服务器按类别组织的特征集合。其作用是在正式训练阶段为客户端提供额外的类别判别信息，缓解本地数据分布偏斜造成的训练偏移。

4.4.1 客户端上传机制

客户端首先从本地数据中提取性能敏感特征，并保留其对应类别标签。上传前，系统对上传内容进行筛选，以控制通信量和共享池规模。

上传限制包括两个层面：一是每个客户端的上传总量上限，二是每个类别的上

传数量上限。前者避免单个客户端上传过多特征，后者避免共享池被部分类别主导。客户端最终上传的内容包括性能敏感特征及其对应类别标签。

4.4.2 服务器共享样本池维护

服务器接收客户端上传的特征和标签后，将其加入共享样本池。共享池先保存所有客户端上传的样本，再按照类别进行整理。每个类别对应一个特征集合，例如类别 0、类别 1 直到类别 K 分别维护独立的样本视图。

按类别维护共享池有两个目的。第一，使服务器能够清楚掌握共享特征的类别分布。第二，使客户端在正式训练阶段可以按类别采样共享特征，从而获得更均衡的类别信息。

为控制存储开销，服务器对共享池设置容量限制。容量控制同样包括两个层面：全局容量上限和单类别容量上限。当共享池总量或某一类别的特征数量超过限制时，服务器会移除较早加入的特征，只保留当前容量范围内的共享样本。

4.4.3 正式训练阶段使用方式

共享样本池构建完成后，插件在正式联邦训练阶段用于增强客户端本地训练。该阶段仍以 FedAvg 为主训练框架，服务器每轮选择部分客户端，下发全局模型参数，同时下发共享样本池中的性能敏感特征及其标签。客户端使用本地数据和共享样本共同训练主分类模型。

设主分类模型为 $F(\cdot)$ ，客户端本地样本为 (x, y) ，服务器下发的共享性能敏感特征及其标签为 $(\tilde{x}_s, \tilde{y})$ 。正式训练阶段的本地目标函数为：

$$\mathcal{L}_{train} = \frac{1}{3}CE(F(x), y) + \frac{2}{3}CE(F(\tilde{x}_s), \tilde{y})$$

其中，第一项为本地数据分类损失，第二项为共享敏感特征分类损失。共享样本不替代本地数据，而是作为辅助输入提供跨客户端类别判别信息。该设计能够缓解客户端本地数据类别分布偏斜，使本地模型更新方向更接近全局任务目标。

该阶段不改变 FedAvg 的服务器聚合规则。客户端训练结束后仍上传主分类模型参数和样本数量，服务器仍按照样本数量进行加权平均。插件的作用集中在本地训练目标中，即通过共享性能敏感特征提高客户端训练数据的有效类别覆盖。

4.5 本章小结

本章介绍了 FedFed 特征蒸馏插件的设计与实现。该插件基于图像空间特征分离思想，将客户端本地图像划分为性能鲁棒特征和性能敏感特征，并通过生成器、分类器和共享样本池实现跨客户端的信息共享。系统采用两阶段训练机制，先进行特征蒸馏和共享样本池构建，再在正式 FedAvg 训练中利用共享特征辅助本地模型训练。该插件在保持 FedAvg 主流程不变的前提下，为异构联邦学习提供了额外的特征共享能力。

第 5 章 实验设计与性能评估

5.1 实验目的

本章通过 CIFAR-10 数据集验证 FedFed 插件在联邦学习图像分类任务中的有效性。实验围绕三个问题展开：第一，FedFed 插件是否能够在 FedAvg 基线上带来性能提升；第二，在不同数据异构程度和本地训练强度下，插件是否保持稳定作用；第三，插件中的共享特征池规模和蒸馏阶段训练强度是否会影响最终性能。

实验以 FedAvg 作为基础联邦学习算法，将 FedFed 作为可选插件接入同一训练框架。未启用插件时，系统执行标准 FedAvg；启用插件时，系统先进行特征蒸馏与共享特征池构建，再在正式联邦训练阶段利用共享敏感特征辅助本地模型更新。两种方法的服务器聚合规则保持一致，从而保证对比重点集中在插件带来的辅助训练效果上。

5.2 实验环境与基础设置

实验数据集采用 CIFAR-10。该数据集包含 10 个图像类别，训练集包含 50000 张图像，测试集包含 10000 张图像。主分类模型采用 ResNet-18。联邦系统设置 10 个客户端，每轮选取 50% 客户端参与训练。正式训练最大通信轮数为 300 轮，本地优化器采用 SGD，学习率为 0.01，动量为 0.9，权重衰减为 0.0001。

数据划分采用 Dirichlet 分布构造标签分布偏斜，并启用客户端样本数量偏斜。Dirichlet 参数 α 用于控制数据异构强度， α 越小，客户端之间的类别分布差异越强。默认设置为 $\alpha=0.1$ ，对应较强的 Non-IID 场景。

5.3 评价指标与公平性控制

FedAvg 在 Non-IID 数据场景下容易受到客户端类别分布偏斜影响。由于不同客户端只持有部分类别或类别比例明显不同，本地模型更新方向可能偏离全局分类目标，最终导致全局模型收敛不稳定或测试精度下降。FedFed 插件的设计动机是在不改变 FedAvg 主训练流程的前提下，为客户端提供额外的跨客户端类别判别信息，从而缓解数据异构造成的性能退化。

本节围绕 FedFed 插件是否有效这一问题展开实验验证。实验将关闭插件的

FedAvg 作为无插件基线，将开启 FedFed 插件的方法作为插件方法。两组方法保持数据集、客户端数量、采样比例、模型结构、优化器和聚合规则一致，区别仅在于是否开启 FedFed 插件。通过这种设置，可以将性能差异主要归因于插件本身。

本节实验从五个方面验证 FedFed 插件的作用。第一，进行主性能对比，验证在强异构条件下开启插件是否能够显著提升 FedAvg 性能。第二，改变 Dirichlet 参数 α ，分析数据异构程度变化时插件收益是否稳定。第三，增大本地训练轮数，观察客户端本地漂移增强时插件是否仍然有效。第四，改变共享特征池容量，分析共享敏感特征数量对性能的影响。第五，改变特征蒸馏训练量，分析蒸馏阶段是否存在充分性和边际收益问题。

本文使用以下指标进行评价。BestAcc 表示训练过程中全局模型在测试集上达到的最高准确率，用于衡量方法的最佳性能；FinalAcc 表示训练结束或早停时的测试准确率，用于衡量最终稳定性能；BestRound 表示取得最高准确率的通信轮次；FinalRound 表示训练停止时的通信轮次；Gain 表示插件方法相对于无插件基线的 BestAcc 提升。

为保证实验对比公平，后续实验在相同数据划分、客户端数量、客户端采样比例、主模型结构、优化器设置和服务端聚合规则下进行。除目标变量外，其余实验条件保持一致。

5.4 FedFed 插件主性能验证

5.4.1 主性能对比

主性能对比采用较强数据异构设置。Dirichlet 参数 $\alpha=0.1$ ，本地训练轮数 $E=1$ 。其中， α 用于控制客户端标签分布偏斜程度，数值越小表示数据异构越强； E 表示每轮通信中客户端在本地数据上训练的 epoch 数。

表 5-1 强异构场景下的主性能对比

方法	α	E	BestAcc	FinalAcc	BestRound	FinalRound	Gain
无插件基线	0.1	1	63.89%	53.83%	187	222	-
FedFed 插件	0.1	1	88.95%	88.53%	165	167	+25.06%

表中， α 表示 Dirichlet 标签分布参数，E 表示本地训练轮数，BestAcc 表示训练过程最高测试准确率，FinalAcc 表示训练结束时测试准确率，BestRound 表示取得最高准确率的通信轮次，FinalRound 表示实际结束轮次，Gain 表示 FedFed 插件相对于无插件基线的最高准确率提升。由表 5-1 可知，在 $\alpha=0.1$ 的强异构场景下，FedFed 插件将 BestAcc 从 63.89% 提升到 88.95%，将 FinalAcc 从 53.83% 提升到 88.53%，说明开启插件后模型性能和训练稳定性均得到明显改善。

图 5-1 强异构场景下主性能对比

图 5-1 对比了两种方法的最高准确率和最终准确率。无插件基线的最终准确率明显低于最高准确率，说明训练后期存在性能回落；FedFed 插件的 BestAcc 与 FinalAcc 接近，说明插件不仅提升了性能上限，也改善了训练稳定性。

图 5-2 强异构场景下的收敛曲线

图 5-2 展示了训练过程中测试准确率随通信轮次的变化。FedFed 插件在较早轮次达到较高精度，并在后续训练中保持稳定；无插件基线虽然中途达到一定精度，但后期波动和下降更明显。这说明 FedFed 插件能够缓解 Non-IID 条件下 FedAvg 的收敛不稳定问题。

5.4.2 数据异构强度分析

为进一步分析插件收益与数据异构程度之间的关系，本文设置 $\alpha=0.3$ 、 $\alpha=0.1$ 和 $\alpha=0.05$ 三种数据划分。 $\alpha=0.3$ 表示中等异构， $\alpha=0.1$ 表示较强异构， $\alpha=0.05$ 表示更强的类别分布偏斜。其余训练参数保持一致。

表 5-2 不同数据异构强度下的性能对比

α	E	方法	BestAcc	FinalAcc	Gain
0.3	1	无插件基线	71.95%	67.81%	—
0.3	1	FedFed 插件	91.98%	91.14%	+20.03%

0.1	1	无插件基线	63.89%	53.83%	-
0.1	1	FedFed 插件	88.95%	88.53%	+25.06%
0.05	1	无插件基线	38.15%	34.93%	-
0.05	1	FedFed 插件	84.80%	78.64%	+46.65%

表中, α 表示数据异构强度控制参数, E 表示本地训练轮数, Gain 表示相同 α 和 E 下 FedFed 插件相对于无插件基线的 BestAcc 提升。由表 5-2 可知, 随着 α 变小, 无插件基线性能快速下降; 相比之下, FedFed 插件在三种异构强度下均保持较高准确率, 并在 $\alpha=0.05$ 时取得 46.65 个百分点的最大提升。

图 5-3 不同数据异构强度下的性能对比

图 5-3 显示, FedFed 插件在不同 α 下均明显优于无插件基线。无插件基线对 α 的变化更加敏感, 而 FedFed 插件的准确率下降幅度较小, 说明共享敏感特征能够提高模型对数据异构的鲁棒性。

图 5-4 不同数据异构强度下的插件增益

图 5-4 展示了 FedFed 插件在不同异构强度下的准确率增益。随着 α 降低, 插件增益整体增大, 说明客户端数据越偏斜, 共享敏感特征提供的跨客户端补充信息越重要。

5.4.3 本地训练强度分析

本地训练轮数增大时, 客户端会在本地数据上执行更多更新。该设置能够减少通信次数, 但在 Non-IID 场景下也可能加剧客户端更新方向差异, 形成更明显的 client drift。为验证 FedFed 插件是否能够缓解这一问题, 本文将本地训练轮数设置为 $E=5$, 并与无插件基线进行对比。

表 5-3 本地训练强度增大时的性能对比

α	E	方法	BestAcc	FinalAcc	BestRound	FinalRound	Gain
0.1	5	无插件基线	59.60%	58.57%	58	120	-
0.1	5	FedFed 插件	91.90%	91.01%	132	167	+32.30%

表中，E 表示每轮通信中的本地训练 epoch 数。较大的 E 会增强本地训练强度，也更容易放大 Non-IID 数据下的客户端漂移。由表 5-3 可知，当 E=5 时，无插件基线最高准确率为 59.60%，而 FedFed 插件达到 91.90%，提升 32.30 个百分点，说明插件能够在本地漂移增强时继续保持有效。

图 5-5 本地训练强度增大时的性能对比

图 5-5 显示，在 E=5 的设置下，FedFed 插件的最高准确率和最终准确率均明显高于无插件基线。这说明共享敏感特征不仅能够补充类别信息，还能在一定程度上约束客户端本地训练方向，使本地更新更接近全局任务目标。

5.5 主性能改进历程分析

5.5.1 初始插件版本

5.5.2 图像空间特征蒸馏版本

5.5.3 共享特征池优化版本

5.5.4 最终插件版本

5.6 插件关键模块消融实验

5.6.1 共享特征池规模消融

共享特征池用于保存各客户端上传的性能敏感特征，并在正式训练阶段下发给客户端辅助训练。共享池容量过小可能导致类别覆盖不足，容量较大则能够保留更多跨客户端判别信息。为分析共享池规模对插件性能的影响，本文设置低容量共享

池、中等容量共享池和不设额外容量限制三种配置。

表 5-4 不同共享特征池规模下的性能对比

共享池设置	上传总量上限	单类上传上限	池总量上限	单类池容量	BestAcc	FinalAcc
低容量共享池	100	4	800	80	80.31%	75.30%
中等容量共享池	1000	20	4000	400	80.12%	75.48%
不设额外容量限制	不限制	不限制	不限制	不限制	90.33%	89.50%

表中，上传总量上限表示单个客户端最多上传的共享特征数量，单类上传上限表示每个类别最多上传的特征数量，池总量上限表示服务器共享池保存的最大特征数量，单类池容量表示服务器对每一类共享特征的容量限制。由表 5-4 可知，不设额外容量限制时，FedFed 插件最高准确率达到 90.33%，明显高于低容量和中等容量设置，说明共享特征池容量是影响插件效果的重要因素。

图 5-6 不同共享特征池规模下的性能对比

图 5-6 显示，不设额外容量限制的共享池在 BestAcc 和 FinalAcc 上均明显优于受限共享池。说明在强异构条件下，FedFed 插件需要足够丰富的共享敏感特征来覆盖客户端缺失类别，过小的共享池会削弱插件作用。

5.6.2 特征蒸馏充分性消融

特征蒸馏阶段决定共享敏感特征的质量。若蒸馏不足，提取出的敏感特征可能缺少稳定判别信息；若蒸馏训练量过大，则可能增加训练开销，并不一定继续提高正式训练性能。本文通过改变蒸馏通信轮数 R_d 和本地蒸馏轮数 E_d 分析蒸馏充分性。

需要说明的是，本组实验中特征蒸馏损失权重 λ_{fd} 保持为 2.0，重构损失权重 λ_{rec} 保持为 5.0，原图分类损失权重 λ_x 保持为 0.4。因此，本组实验主要分析蒸馏训练量变化，而不是蒸馏损失权重变化。

表 5-5 不同特征蒸馏训练量下的性能对比

蒸馏设置	λ_{fd}	R_d	E_d	BestAcc	FinalAcc	BestRound
短程蒸馏	2.0	5	1	89.73%	89.43%	142
标准蒸馏	2.0	15	1	88.41%	87.50%	117
长程蒸馏	2.0	30	1	89.26%	88.73%	157
长程增强蒸馏	2.0	30	2	90.30%	88.19%	212

表中， λ_{fd} 表示特征蒸馏分类损失权重， R_d 表示特征蒸馏阶段的通信轮数， E_d 表示每轮蒸馏中的本地训练 epoch 数。 R_d 和 E_d 越大，表示蒸馏阶段训练量越大。由表 5-5 可知，不同蒸馏训练量下插件均能取得较高准确率，但性能并不随 R_d 或 E_d 增大而单调提升。

图 5-7 不同特征蒸馏训练量下的性能对比

图 5-7 显示，不同蒸馏训练量下的性能差异相对有限，且不存在明显单调趋势。这说明蒸馏阶段的主要作用是获得可用的敏感特征表示；当特征分离质量达到一定程度后，继续增加蒸馏训练量的边际收益有限。

5.6.3 插件小模块消融一

5.6.4 插件小模块消融二

5.6.5 插件小模块消融三

5.6.6 插件小模块消融四

5.7 本章小结

本章围绕 FedFed 插件的性能表现展开实验评估。首先给出实验目的、基础设置和评价指标；随后从主性能、数据异构强度和本地训练强度三个角度验证插件有效性；最后通过性能改进历程和关键模块消融分析不同设计对最终结果的影响。

第 6 章 机制验证与工程分析

6.1 特征蒸馏机制诊断

为进一步解释 FedFed 插件的性能来源，本文对特征蒸馏阶段得到的不同输入形式进行分类诊断。该实验不直接比较联邦训练最终精度，而是检验原始图像、性能敏感特征、性能鲁棒特征以及带噪敏感特征各自保留的任务信息。

实验设置六类分类代理任务。其中， $x \rightarrow x$ 表示使用原始图像训练并在原始图像上测试； $x_s \rightarrow x_s$ 表示使用性能敏感特征训练并在性能敏感特征上测试； $x_r \rightarrow x_r$ 表示使用性能鲁棒特征训练并在性能鲁棒特征上测试； $x_s \rightarrow x$ 表示使用性能敏感特征训练并迁移到原始图像测试； $x+x_s \rightarrow x$ 表示同时使用原始图像和敏感特征训练，并在原始图像上测试； $x+x_p \rightarrow x$ 表示使用原始图像和带噪敏感特征训练，并在原始图像上测试。

表 6-1 特征蒸馏机制诊断结果

代理任务	输入含义	BestAcc	FinalAcc	BestEpoch
$x \rightarrow x$	原始图像分类	69.55%	69.55%	10
$x_s \rightarrow x_s$	敏感特征分类	74.00%	74.00%	10
$x_r \rightarrow x_r$	鲁棒特征分类	54.23%	53.53%	7
$x_s \rightarrow x$	敏感特征到原图迁移	14.29%	14.29%	10
$x+x_s \rightarrow x$	原图与敏感特征联合训练	73.58%	71.29%	8
$x+x_p \rightarrow x$	原图与带噪敏感特征联合训练	71.28%	71.22%	7

表中，BestAcc 表示代理分类器训练过程中的最高测试准确率，FinalAcc 表示训练结束时的测试准确率，BestEpoch 表示取得最高准确率的训练轮次。 x_s 表示性能敏感特征， x_r 表示性能鲁棒特征， x_p 表示加入扰动后的性能敏感特征。

由表 6-1 可知， $x_s \rightarrow x_s$ 的最高准确率达到 74.00%，高于原始图像代理任务 $x \rightarrow x$ 的 69.55%，说明性能敏感特征中保留了较强的分类判别信息。相比之下， $x_r \rightarrow x_r$ 的最高准确率为 54.23%，明显低于 $x_s \rightarrow x_s$ ，说明性能鲁棒特征中的分类信

息相对较弱。该结果符合 FedFed 的特征蒸馏目标：敏感特征主要承载与分类任务相关的信息，鲁棒特征更多保留非关键的隐私信息。

同时， $x+x_s \rightarrow x$ 的最高准确率达到 73.58%，高于单独使用原始图像的 $x \rightarrow x$ ，说明敏感特征可以作为有效的辅助训练信息。加入扰动后的 $x+x_p \rightarrow x$ 仍取得 71.28% 的最高准确率，表明扰动后的敏感特征仍然保留任务可用性。结合前文隐私验证结果可知，带噪敏感特征在保持分类辅助作用的同时，可以降低直接反演原始图像的风险。

图 6-1 不同特征输入形式下的分类诊断结果

图 6-1 对比了原始图像、敏感特征、鲁棒特征和带噪敏感特征的分类代理性能。结果显示，敏感特征具有较强分类能力，鲁棒特征分类性较弱，说明 FedFed 插件的特征蒸馏过程能够将更多分类相关信息集中到可共享特征中。

6.2 隐私验证

FedFed 插件通过共享性能敏感特征缓解客户端数据异构问题，但共享特征本身仍可能带来隐私泄露风险。为验证本文插件在共享特征过程中的隐私表现，本节从敏感特征范数、模型反演攻击和成员推断攻击三个角度进行实验分析。

6.2.1 隐私验证思路

本文中特征蒸馏阶段将原始图像 x 分解为性能鲁棒特征 x_r 和性能敏感特征 x_s ，其中：

$$x_s = x - x_r$$

性能敏感特征 x_s 包含较强的分类判别信息，因此可以用于辅助联邦训练。但如果直接共享 x_s ，攻击者可能通过反演攻击恢复原始图像，或通过成员推断攻击判断某个样本是否参与过训练。

因此，本文在共享前对 x_s 引入 L2 范数剪裁和高斯噪声扰动。具体做法为：首先对每个样本的敏感特征进行 L2 范数剪裁，将其范数限制在阈值 C 内；随后参考 FedFed 原论文中的噪声注入方式，对剪裁后的敏感特征加入高斯噪声，得到带噪敏

敏感特征 x_p 。

本文使用 $x+x_p \rightarrow x$ 的分类代理任务评估带噪敏感特征的可用性。这里的 $x+x_p$ 并不是指将原始图像和带噪敏感特征逐元素相加，而是指在分类器训练时同时使用原始图像 x 和带噪敏感特征 x_p 作为训练样本，并在测试时使用原始图像 x 进行分类评估。若该分类器仍能取得较高准确率，说明 x_p 中保留了对分类任务有用的信息，能够作为共享辅助特征参与训练。

6.2.2 敏感特征剪裁与噪声设置

在确定剪裁与噪声参数前，本文首先统计蒸馏阶段得到的敏感特征 x_s 的 L2 范数分布。该实验的目的是观察 x_s 的实际数值规模，避免在不了解特征范数的情况下直接设置剪裁阈值或噪声强度。若剪裁阈值过小，会破坏大量有效特征；若阈值过大，则少量异常大范数样本可能削弱固定噪声的扰动效果。

图 6-2 展示了不同配置下敏感特征 x_s 的 L2 范数统计结果。

图 6-2 敏感特征 L2 范数统计对比

实验结果显示，在无剪裁无噪声配置下，测试集 x_s 的平均范数约为 8.30，p95 为 11.61，p99 为 13.29，最大值达到 23.23。这说明大部分敏感特征集中在 10 左右，但仍存在少量明显偏大的样本。因此，本文选择 $C=10$ 作为剪裁阈值。该阈值能够覆盖主要样本分布，同时限制高范数异常样本。

采用 $C=10$ 后，测试集 x_s 的 p95、p99 和最大范数均被限制在 10 附近，说明剪裁机制按预期生效。与此同时，最佳配置下 x_p 的平均范数约为 11.68，p95 约为 13.09，表明噪声扰动后特征规模仍处于可控范围内，没有出现明显数值失控。

在噪声强度选择上，本文对多组高斯噪声配置进行了对比。实验发现， $C=10$ 、Gaussian(0.15, 0.20) 在分类代理准确率和反演风险之间取得较好折中：相较无剪裁无噪声配置，分类代理准确率仅下降约 1.22 个百分点，同时反演 PSNR 从 18.87 降至 16.58。因此，本文将该组参数作为后续分析的最佳配置。

最佳配置为：

$$C = 10, \quad \sigma_1 = 0.15, \quad \sigma_2 = 0.20$$

其中, C 为敏感特征 L2 范数剪裁阈值, σ_1 和 σ_2 为两路高斯噪声标准差。本文采用两路噪声设置, 主要参考 FedFed 原论文中对敏感特征加入两种强度扰动的思想。其基本考虑是: 较弱噪声用于保留敏感特征中的主要判别信息, 使带噪特征仍能辅助分类训练; 较强噪声用于进一步增强共享阶段的扰动强度, 降低敏感特征被直接恢复的风险。通过设置两种噪声强度, 系统能够在特征可用性和隐私扰动之间进行折中, 而不是只依赖单一噪声强度。

在本文实验中, σ_1 主要用于构造隐私验证中的带噪敏感特征, σ_2 用于对应共享阶段更强扰动设置, 二者共同构成本文对 FedFed 噪声扰动思想的实现。

6.2.3 模型反演攻击验证

模型反演攻击用于评估攻击者能否从共享特征中恢复原始图像。本文训练一个反演网络, 以带噪敏感特征 x_p 为输入, 以原始图像 x 为重建目标, 并使用 MSE、L1 误差和 PSNR 衡量重建质量。

其中, MSE 表示重建图像与原始图像之间的均方误差, 数值越大表示重建误差越大; L1 误差表示像素级绝对误差的平均值, 数值越大表示重建结果与原图差异越大; PSNR 表示峰值信噪比, 数值越高表示重建图像越接近原图, 数值越低表示反演恢复越困难。分类代理准确率 $x+x_p \rightarrow x$ Acc 用于衡量带噪敏感特征对分类任务的可用性, 数值越高表示特征仍保留较强任务信息。

表 6-2 模型反演攻击结果

配置	C	噪声设置	$x+x_p \rightarrow x$ Acc	MSE	L1	PSNR
无剪裁无噪声	-	None	63.33%	0.01296	0.08661	18.87
最佳配置	10	Gaussian(0.15,0.20)	62.10%	0.02196	0.11201	16.58

由表 6-2 可知, 最佳配置下分类代理准确率由 63.33% 下降至 62.10%, 仅下降约 1.22 个百分点, 说明剪裁与噪声对特征可用性的影响较小。同时, 模型反演 MSE 由 0.01296 增大至 0.02196, L1 误差由 0.08661 增大至 0.11201, PSNR 由 18.87 降低至 16.58。三项反演指标均表明, 攻击者从带噪敏感特征中恢复原始图像的难度明显增加。

为进一步直观观察反演攻击效果，本文分别给出无剪裁无噪声配置和最佳配置下的模型反演可视化结果，如图 6-3 和图 6-4 所示。

图 6-3 无剪裁无噪声配置下的模型反演结果

图 6-4 最佳配置下的模型反演结果

图中每组样本依次展示原始图像、共享特征、反演重建图像和重建误差。无剪裁无噪声配置下，反演网络能够从共享敏感特征中恢复出较明显的图像结构和颜色分布；而在最佳配置下，反演重建图像更加模糊，局部细节和类别相关视觉结构明显减弱，重建误差更加突出。该现象与表 6-2 中的定量结果一致，说明剪裁与噪声扰动降低了共享敏感特征的可反演性。

6.2.4 成员推断攻击验证

成员推断攻击用于判断某个样本是否参与过目标模型训练。本文采用基于影子模型的成员推断攻击，即 Shadow Model-based Membership Inference Attack。该攻击假设攻击者无法直接获得目标训练集，但可以获得与目标任务分布相似的辅助数据，并能够查询目标模型输出。在联邦学习场景中，攻击者可以被建模为潜在恶意客户端，其能够获得服务器下发的全局模型，并利用自身持有的同分布数据训练影子模型。

具体流程如下。攻击者首先将辅助数据划分为 shadow train 和 shadow test，其中 shadow train 用于训练影子模型，shadow test 不参与影子模型训练。由于影子模型由攻击者自行训练，因此 shadow train 样本被标记为 member，shadow test 样本被标记为 non-member。随后，攻击者使用与目标模型相同的输入形式训练多个影子模型，并将样本输入影子模型，提取模型输出中的 top-k 概率、置信度、预测间隔、熵、真实类别概率和交叉熵损失等统计特征。最后，攻击者使用这些统计特征训练二分类攻击模型，并将该攻击模型迁移到目标模型上进行成员推断。

为避免攻击模型过弱导致低估成员推断风险，本文进一步增强 Shadow MIA 设置，包括增加 shadow model 数量、增加影子模型训练轮数，并扩大成员推断评估样

本数量。增强后的实验设置和结果如表 6-3 所示。

表 6-3 增强 Shadow MIA 设置下的成员推断攻击结果

配置	Shadow Models	Shadow Epochs	MIA Samples	MIA AUC	Attack Acc	Member Recall	Member Precision
无剪裁无噪声	5	20	1500	0.702	63.43%	96.80%	58.06%
最佳配置	5	20	1500	0.769	69.17%	97.73%	62.20%

其中，Shadow Models 表示训练的影子模型数量；Shadow Epochs 表示每个影子模型的训练轮数；MIA Samples 表示用于目标模型成员推断评估的成员样本和非成员样本数量。MIA AUC 衡量攻击模型区分成员样本和非成员样本的整体能力，越接近 0.5 表示越接近随机猜测，越接近 1 表示攻击能力越强。Attack Acc 表示攻击模型最终二分类准确率；Member Recall 表示真实成员样本中被识别为成员的比例；Member Precision 表示被攻击模型判断为成员的样本中真实成员所占比例。

从增强攻击结果看，无剪裁无噪声配置下 MIA AUC 为 0.702，说明在较强影子模型攻击设置下，攻击者已经能够获得一定成员区分能力。最佳配置下 MIA AUC 进一步升至 0.769，Attack Acc 由 63.43% 提高至 69.17%。该结果表明，剪裁与噪声扰动虽然降低了模型反演风险，但并未稳定降低成员推断风险。

造成这一现象的原因在于，模型反演攻击和成员推断攻击关注的泄露形式不同。模型反演攻击关注能否从共享特征恢复原始图像，而成员推断攻击关注模型对成员样本和非成员样本的输出差异。加入固定噪声后，带噪敏感特征的图像细节被破坏，因此反演难度增加；但固定带噪特征也可能使模型对训练样本产生更明显的输出记忆，从而增大成员样本与非成员样本在置信度、熵和损失等统计特征上的差异。因此，本文认为该剪裁与噪声机制主要降低共享特征的反演风险，而不能视为完整的成员隐私保护方法。

6.2.5 隐私验证结果分析

图 6-5 汇总展示了不同配置下的分类代理准确率、MIA AUC 和模型反演 PSNR。

图 6-5 不同配置下的性能与隐私指标对比

由图可知，最佳配置在分类代理准确率上仅有小幅下降，说明其仍保留较好的特征可用性；同时模型反演 PSNR 明显降低，说明其能够削弱共享特征中的可恢复图像信息。但是，MIA AUC 未稳定下降，说明剪裁与噪声并不能完整解决成员推断风险。

综合上述实验，可以得到以下结论。

第一，敏感特征剪裁能够有效限制异常大范数特征。无剪裁时测试集 x_s 最大范数达到 23.23，而采用 $C=10$ 后， x_s 的 p95、p99 和最大范数均被限制在 10 附近，说明剪裁操作能够稳定约束敏感特征规模。

第二，剪裁与高斯噪声能够在较小影响分类代理性能的情况下明显降低模型反演风险。最佳配置下分类代理准确率仅下降约 1.22 个百分点，而反演 PSNR 从 18.87 降至 16.58，说明共享特征中的可恢复图像信息减少。

第三，剪裁与噪声对成员推断攻击的防护效果有限。实验发现，固定带噪敏感特征可能使模型对训练样本和测试样本产生更明显的输出差异，从而导致 MIA AUC 升高。因此，本文不将该机制视为严格的成员隐私保护方法。

综上，本文的隐私增强机制主要用于降低共享敏感特征的可反演性，而不是严格差分隐私机制。实验表明，该方法能够在保持特征可用性的同时降低模型反演风险，但对于成员推断风险仍需结合正则化训练、动态噪声扰动或差分隐私训练等方法进一步改进。该结论与本文工作定位一致，即在 FedFed 特征蒸馏思想基础上进行插件化实现，并对共享特征的隐私风险进行经验性验证。

6.3 跨联邦优化器可插拔验证

前述实验主要基于 FedAvg 主流程验证 FedFed 插件的有效性。为进一步验证插件设计的可插拔性，本文将 FedFed 插件接入 FedProx、SCAFFOLD、FedAvgM 和 FedNova 四种联邦优化器，比较开启插件前后的分类性能。该实验用于验证插件接口能否在不同联邦优化主流程中复用，并分析插件效果与底层优化器之间的关系。

表 6-4 不同联邦优化器下的插件接入结果

基础优化器	插件状态	BestAcc	FinalAcc	BestRound	FinalRound	Gain
FedProx	关闭	59.91%	58.65%	133	168	-
FedProx	开启	90.49%	90.25%	279	300	+30.58%
SCAFFOLD	关闭	51.68%	40.33%	161	196	-
SCAFFOLD	开启	56.27%	53.54%	172	207	+4.59%
FedAvgM	关闭	49.52%	43.87%	140	175	-
FedAvgM	开启	80.20%	70.86%	227	262	+30.68%
FedNova	关闭	47.04%	42.73%	64	120	-
FedNova	开启	70.77%	62.87%	172	207	+23.73%

表中，基础优化器表示联邦训练使用的主优化方法，插件状态表示是否启用 FedFed 插件，Gain 表示同一基础优化器下开启插件后相对于关闭插件的 BestAcc 提升。

由表 6-4 可知，FedFed 插件能够接入多种联邦优化器，并在 FedProx、FedAvgM 和 FedNova 上带来明显性能提升。其中，FedProx 上最高准确率由 59.91% 提升到 90.49%，FedAvgM 上由 49.52% 提升到 80.20%，FedNova 上由 47.04% 提升到 70.77%。该结果说明，本文插件接口不依赖单一 FedAvg 实现，而是可以作为辅助训练模块接入不同联邦优化流程。

SCAFFOLD 上的提升相对有限，BestAcc 由 51.68% 提升到 56.27%。SCAFFOLD 通过服务器端和客户端控制变量校正本地梯度方向，其核心目标是降低 Non-IID 场景下的 client drift；FedFed 插件则通过共享敏感特征改变客户端本地训练数据分布，为本地分类目标提供额外信息。二者都作用于客户端更新方向，但作用路径不同：SCAFFOLD 从梯度校正角度约束更新，FedFed 从数据补充角度改变本地训练信号。当控制变量估计和共享特征辅助目标同时作用时，客户端更新受到两类校正信号共同影响，插件增益被削弱。因此，跨算法实验表明，FedFed 插件具备跨联邦优化器接入能力，但性能收益会随底层优化器机制和训练配置变化。

图 6-6 不同联邦优化器下的插件性能对比

图 6-6 展示了不同基础优化器开启和关闭 FedFed 插件后的最高准确率。可以看到，插件在多数优化器上带来性能提升，但提升幅度并不完全一致，说明插件具有可插拔性，同时其效果与底层联邦优化策略存在耦合关系。

6.4 训练开销分析

FedFed 插件通过特征蒸馏和共享特征辅助训练提升模型性能，但也会带来额外训练开销。为客观评价插件的工程代价，本文统计不同设置下无插件基线和 FedFed 插件的训练时间，并计算相对耗时倍率。

表 6-5 FedFed 插件训练开销对比

实验设置	方法	DistillRounds	共享池设置	BestAcc	Duration	相对倍率
$\alpha=0.1$, $E=1$	无插件基线	0	无	63.89%	44.11 min	1.00×
$\alpha=0.1$, $E=1$	FedFed 插件	15	不设额外限制	88.95%	163.32 min	3.70×
$\alpha=0.1$, $E=5$	无插件基线	0	无	59.60%	116.03 min	1.00×
$\alpha=0.1$, $E=5$	FedFed 插件	15	不设额外限制	91.90%	600.50 min	5.18×

表中，DistillRounds 表示特征蒸馏阶段的通信轮数，Duration 表示完整实验运行时间，相对倍率表示同一实验设置下 FedFed 插件方法相对于无插件基线的耗时倍数。共享池设置为“不设额外限制”表示实验中未额外限制共享特征池总容量和单类容量。

由表 6-5 可知，在 $\alpha=0.1$, $E=1$ 的主实验设置下，FedFed 插件将最高准确率从 63.89% 提升到 88.95%，训练时间由 44.11 min 增加到 163.32 min，约为无插件基线的 3.70 倍。在 $E=5$ 的设置下，FedFed 插件最高准确率达到 91.90%，训练时间增加到 600.50 min，约为无插件基线的 5.18 倍。

该结果说明，FedFed 插件的性能提升伴随额外计算成本。其开销主要来自两部分：一是正式训练前的特征蒸馏阶段；二是正式训练阶段对共享敏感特征的额外训练。当本地训练轮数增大时，共享特征辅助训练的计算量进一步增加。

因此，FedFed 插件适合用于数据异构较强、精度收益需求较高的场景。在实际部署中，需要结合计算资源约束选择蒸馏轮数、共享池规模和本地训练轮数，以在模型性能和训练开销之间取得平衡。

图 6-7 FedFed 插件训练时间开销对比

图 6-7 展示了无插件基线和 FedFed 插件在 $E=1$ 、 $E=5$ 两种设置下的训练时间。FedFed 插件带来明显精度提升的同时，也引入额外计算开销，体现了性能收益与工程成本之间的权衡关系。

6.5 本章小结

本章从机制、安全性、可插拔性和工程开销四个角度对 FedFed 插件进行补充分析。实验结果表明，插件能够将分类相关信息集中到性能敏感特征中，并具备跨联邦优化器接入能力；同时，隐私验证和开销分析说明该方法仍需要在隐私风险控制和训练成本之间进行权衡。

结 论

论文的结论作为论文正文的最后一章单独排写，但不加章标题序号。

结论是对整个论文主要成果的总结。在结论中应明确指出本研究内容的创新性成果或创新点（含新见解、新观点），论文结论应概括论文的核心观点，明确、客观地指出本研究内容的创新点，并指出今后进一步在本作的展望与设想。结论内容一般在 1000 字以内。

参考文献

- [1] **McMahan B., Moore E., Ramage D., et al.** Communication-efficient learning of deep networks from decentralized data[C]// AISTATS. 2017.
- [2] **Kairouz P., McMahan H. B., Avent B., et al.** Advances and Open Problems in Federated Learning[J]. Foundations and Trends in Machine Learning, 2021.
- [3] **Li T., Sahu A. K., Talwalkar A., et al.** Federated Optimization in Heterogeneous Networks[C]// MLSys. 2020. (FedProx)
- [4] **Karimireddy S. P., Kale S., Mohri M., et al.** SCAFFOLD: Stochastic Controlled Averaging for Federated Learning[C]// ICML. 2020.
- [5] **Wang J., Liu Q., Liang H., et al.** Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization[C]// NeurIPS. 2020. (FedNova)
- [6] **Xu J., Zhang X., Liu Y., et al.** FedSPU: Personalized Federated Learning for Resource-Constrained Devices with Stochastic Parameter Update[J]. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- [7] **Liu Y., Wang Y., Li H., et al.** Look Back for More: Harnessing Historical Sequential Updates for Personalized Federated Adapter Tuning[C]// *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023.
- [8] **Zhu Y., Zhang Z., Liu X., et al.** Virtual Nodes Can Help: Tackling Distribution Shifts in Federated Graph Learning[C]// *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2023.
- [9] **Chen Y., Wu L., Zhuang F., et al.** Federated Graph Learning under Domain Shift with Generalizable Prototypes[C]// *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2023.

- [10] **Zhao B., Li X., Liu Z., et al.** FedFed: Feature Distillation against Data Heterogeneity in Federated Learning[C]// *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.
- [11] Kairouz P, McMahan H B, Avenet B, et al. Advances and open problems in federated learning[J]. *Foundations and Trends in Machine Learning*, 2021, 14(1-2): 1-210.
- [12] McMahan B, Moore E, Ramage D, et al. Communication-efficient learning of deep networks from decentralized data[C]// *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. PMLR, 2017: 1273-1282.
- [13] Li T, Sahu A K, Zaheer M, et al. Federated optimization in heterogeneous networks[C]// *Proceedings of Machine Learning and Systems*. 2020, 2: 429-450.
- [14] Wang J, Liu Q, Liang H, et al. Tackling the objective inconsistency problem in heterogeneous federated optimization[C]// *Advances in Neural Information Processing Systems*. 2020, 33: 7611-7623.
- [15] Karimireddy S P, Kale S, Mohri M, et al. SCAFFOLD: Stochastic controlled averaging for federated learning[C]// *International Conference on Machine Learning*. PMLR, 2020: 5132-5143.
- [16] Wang T, Zhu J Y, Torralba A, et al. Dataset distillation[EB/OL]. arXiv:1811.10959, 2018.
- [17] Kingma D P, Welling M. Auto-encoding variational Bayes[C]// *International Conference on Learning Representations*. 2014.
- [18] Yang Z, Zhang Y, Zheng Y, et al. FedFed: Feature distillation against data heterogeneity in federated learning[C]// *Advances in Neural Information Processing Systems*. 2023, 36.
- [19] Yang Z, Zhang Y, Zheng Y, et al. FedFed: Feature distillation against data heterogeneity in federated learning: official implementation[EB/OL]. GitHub, 2023.

致 谢

衷心感谢导师×××教授对本人的精心指导。他的言传身教将使我终生受益。
感谢×××教授，以及实验室全体老师和同窗们的热情帮助和支持！